AIC303 – Λειτουργικά Συστήματα

Εργασία 1: Υλοποίηση ενός απλού φλοιού (shell)

Παράδοση έως και στις 2 Νοεμβρίου 2026 (23:55)

1. Εισαγωγή

Στην παρούσα εργασία θα εξοικειωθούμε με βασικούς μηχανισμούς του λειτουργικού συστήματος Linux, όπως η δημιουργία και εκτέλεση διεργασιών, η επικοινωνία μεταξύ διεργασιών, η ανακατεύθυνση εισόδου/εξόδου και η διαχείριση σημάτων.

Για το σκοπό αυτό, θα υλοποιήσουμε ένα απλό κέλυφος εντολών (*shell*) σε γλώσσα C. Το πρόγραμμά μας θα διαβάζει εντολές από τον χρήστη, θα δημιουργεί τις κατάλληλες διεργασίες και θα εκτελεί τις εντολές αξιοποιώντας τις αντίστοιχες κλήσεις συστήματος του Linux.

Η εργασία έχει σχεδιαστεί έτσι ώστε να υλοποιηθεί σταδιακά. Αρχικά, θα δημιουργήσουμε ένα πρόγραμμα που εκτελεί μία εντολή. Στη συνέχεια, θα επεκτείνουμε τη λειτουργικότητά του ώστε να υποστηρίζει εκτέλεση στο παρασκήνιο, σωληνώσεις (pipelines) και ανακατευθύνσεις εισόδου/εξόδου (I/O redirections).

2. Περιβάλλον εκτέλεσης

Το περιβάλλον στο οποίο θα εκτελέσουμε το παρόν εργαστήριο αποτελείται από μια εικονική μηχανή VirtualBox που έχει εγκατεστημένο το Λειτουργικό Σύστημα Debian 13. Για να εκτελέσουμε την εικονική μηχανή, θα χρειαστούμε το σύστημα εικονικοποίησης Oracle VirtualBox, το οποίο μπορούμε να το κατεβάσουμε δωρεάν για περιβάλλον Windows, Linux, ή MacOS από την Oracle. Θα χρειαστεί αρχικά να κατεβάσουμε την έκδοση που ταιριάζει καλύτερα με το σύστημά μας από τον παρακάτω σύνδεσμο:

https://www.virtualbox.org/wiki/Downloads

Αφού κατεβάσουμε και εγκαταστήσουμε το Oracle VirtualBox μπορούμε να κατεβάσουμε την εικονική μηχανή από τον παρακάτω σύνδεσμο, αφού πρώτα συνδεθούμε στον ιδρυματικό μας λογαριασμό:

https://drive.google.com/file/d/1k_tXfIjFD2MQWpGa_GiQq7ltcMltN45-/view?usp=sharing

Η εικονική μηχανή βρίσκεται στο αρχείο **AIC303.ova**. Αφού κατεβάσουμε το αρχείο στο δίσκο μας, θα χρειαστεί να εισάγουμε την εικονική μηχανή στο VirtualBox επιλέγοντας από το μενού File → Import Appliance. Στη συνέχεια, επιλέγουμε το εικονίδιο του φακέλου και από το παράθυρο που θα ανοίγει εντοπίζουμε και επιλέγουμε το αρχείο AIC303.ova. Τέλος, επιλέγουμε Open.

Η εικονική μηχανή είναι ρυθμισμένη ώστε να χρησιμοποιεί 2 GB μνήμη RAM, 2 πυρήνες επεξεργαστή, και έναν εικονικό δίσκο με μέγιστο μέγεθος 16 GB. Μπορούμε να αυξήσουμε τους πόρους, ανάλογα με τους διαθέσιμους πόρους στο μηχάνημα φιλοξενίας. Το δίκτυο της εικονικής μηχανής έχει ρυθμιστεί με την επιλογή NAT, ώστε να διαμοιράζεται τη σύνδεση δικτύου του συστήματος φιλοξενίας.

Μπορούμε να συνδεθούμε στην εικονική μηχανή είτε ως απλός χρήστης χρησιμοποιώντας ως όνομα χρήστη το “user” και κωδικό πρόσβασης “user”, είτε ως διαχειριστής χρησιμοποιώντας ως όνομα χρήστη το “root” και κωδικό πρόσβασης το “root”. Προτείνεται να συνδεόμαστε στην εικονική μηχανή ως απλός χρήστης.

Στην εικονική μηχανή είναι εγκατεστημένοι οι κειμενογράφοι Codium (Microsoft Visual Studio Code), Mouspad, Geany, και Emacs. Μπορούμε να εγκαταστήσουμε ένα νέο πρόγραμμα, ανοίγοντας ένα τερματικό ως διαχειριστής και δίνοντας τις εντολές:

```
$ apt update ; apt install package-name
```

Για να αναζητήσετε ένα πακέτο χρησιμοποιήστε την εντολή:

```
$ apt search package-name
```

Σημείωση: Μια εναλλακτική επιλογή είναι να εκτελέσουμε την εργασία απευθείας πάνω στον προσωπικό μας υπολογιστή, εφόσον έχετε ήδη εγκατεστημένη κάποια πρόσφατη διανομή Linux.

3. Υλοποίηση του απλού φλοιού minsh

Στην παρούσα εργασία θα υλοποιήσουμε ένα απλό πρόγραμμα φλοιού, το οποίο ονομάζεται **minsh**. Γενικότερα, ένα πρόγραμμα φλοιού, παρέχει μια διεπαφή κειμένου για την επικοινωνία ενός χρήστη με το σύστημα. Η διεπαφή κειμένου δέχεται εντολές, τις εκτελεί, και επιστρέφει το αποτέλεσμά τους. Υπάρχουν πολλά προγράμματα φλοιού, όπου το κάθε ένα έχει διαφορετική σύνταξη εντολών. Στο Linux, το βασικό πρόγραμμα φλοιού λέγεται Bash.

Μια γραμμή εντολής έχει τη μορφή:

```
minish$ command argument_1 argument2 …
```

Η πρώτη λέξη είναι η προτροπή εισόδου, η οποία μπορεί να περιλαμβάνει το όνομα του προγράμματος φλοιού, καθώς και άλλες πληροφορίες όπως το όνομα χρήστη με το οποίο έχουμε συνδεθεί στο σύστημα και τον τρέχων φάκελο εργασίας. Η δεύτερη λέξη είναι η εντολή που θα εκτελεστεί, ακολουθεί ένα κενό, και στη συνέχεια ακολουθεί μια προαιρετική λίστα από ορίσματα. Η γραμμή εντολής τερματίζεται με το χαρακτήρα αλλαγής γραμμής **‘\n’**, όταν ο χρήστης πιέζει το πλήκτρο Enter.

3.1. Προετοιμασία

Αφού συνδεθούμε στην εικονική μηχανή, κατεβάζουμε από το Github του μαθήματος το αποθετήριο **minsh-base**, στο οποίο θα βασιστεί η υλοποίησή μας.

Ο κώδικας του minsh υπάρχει διαθέσιμος στον παρακάτω σύνδεσμο στο Github:

https://github.com/OS-AIC303/minish-base

Για να υλοποιήσουμε την εργασία, θα χρειαστεί να κλωνοποιήσουμε το αποθετήριο, να υλοποιήσουμε τις λειτουργίες που λείπουν, και να παραδώσουμε τη λύση μας, σύμφωνα με τις οδηγίες που βρίσκονται στην τελευταία ενότητα. Για να κλωνοποιήσουμε το αποθετήριο ανοίγουμε ένα τερματικό και εκτελούμε:

```
$ git clone https://github.com/OS-AIC303/minish-base.git
```

Μετά την εκτέλεση της παραπάνω εντολής θα δημιουργηθεί κάτω από τον τρέχων φάκελο εργασίας μας ο φάκελος *minsh-base*, ο οποίος περιλαμβάνει μια βασική υλοποίηση του φλοιού minsh.

3.2. Δομή

Εσωτερικά στο φάκελο *minsh-base*, μπορούμε να βρούμε το αρχείο *Makefile*, το οποίο αποτελεί μια “συνταγή” για τη μεταγλώττιση του φλοιού. Ο φάκελος *include* περιλαμβάνει το αρχείο κεφαλίδας *parser.h*. Ο φάκελος *src* περιλαμβάνει τον πηγαίο κώδικα του φλοιού στο αρχείο *shell.c*, καθώς και τον πηγαίο κώδικα ενός αναλυτή στο αρχείο *parser.c*. Στο φάκελο *build*, δημιουργείται το εκτελέσιμο του φλοιού μας, κάθε φορά που εκτελούμε μια εντολή μεταγλώττισης.

Κάθε φορά που τροποποιούμε τον κώδικα του minsh θα πρέπει να τον μεταγλωττίζουμε, εκτελώντας σε ένα τερματικό την εντολή make, αφού πρώτα εισέλθουμε στον φάκελο minish-base:

```
$ cd minish-base
```

```
$ make
```

Μπορούμε να καθαρίσουμε το αποτέλεσμα της προηγούμενης μεταγλώττισης εκτελώντας:

```
$ make clean
```

Αφού μεταγλωττίσουμε το φλοιό μας, μπορούμε να τον εκκινήσουμε εκτελώντας την εντολή:

```
$ ./build/minsh
```

3.3. Ανάλυση γραμμής εντολής

Μια βασική λειτουργία του φλοιού είναι το διάβασμα μιας γραμμής εντολής και η ανάλυσή της. Το αποτέλεσμα της διαδικασίας αυτής είναι μια δομή που περιλαμβάνει την εντολή (ή τις εντολές) που περιλαμβάνει η τρέχουσα γραμμή εντολών, πιθανά ορίσματα για κάθε εντολή, και πιθανούς τελεστές.

Για διευκόλυνση, το minsh περιλαμβάνει μια **έτοιμη υλοποίηση** ενός βασικού αναλυτή (parser), ο οποίος διαβάζει μια γραμμή εντολής από το πληκτρολόγιο, την αναλύει, και δημιουργεί τις απαραίτητες δομές για κάθε εντολή που εντοπίζει. Έτσι, **εμείς δε χρειάζεται να κάνουμε απολύτως καμία αλλαγή στον αναλυτή**, αλλά **μπορούμε να χρησιμοποιήσουμε τη διεπαφή που μας προσφέρει έτοιμη**, την οποία θα βρούμε στο αρχείο *parser.h*.

Η βασική συνάρτηση της διεπαφής είναι η `parse_line()`, η οποία δέχεται ως είσοδο μια συμβολοσειρά που περιέχει τη γραμμή εντολών του χρήστη και έναν δείκτη σε μια δομή *pipeline*, στην οποία αποθηκεύονται τα αποτελέσματα της ανάλυσης. Η συνάρτηση επιστρέφει έναν ακέραιο που υποδεικνύει αν η ανάλυση ολοκληρώθηκε επιτυχώς (τιμή 0) ή αν παρουσιάστηκε κάποιο σφάλμα (αρνητικός αριθμός). Η δομή *pipeline* αποτελεί το βασικό αντικείμενο που επιστρέφεται έμμεσα από τον parser και περιέχει έναν πίνακα εντολών (*commands*), το πλήθος τους (*command_count*), τη χωρητικότητα του πίνακα (*command_capacity*) και την ένδειξη *background*, η οποία προσδιορίζει αν έχει ζητηθεί εκτέλεση στο παρασκήνιο.

Κάθε εντολή αναπαρίσταται από τη δομή *command*. Η δομή αυτή περιλαμβάνει τον πίνακα *argv*, ο οποίος αποθηκεύει το όνομα της εντολής και τα ορίσματά της, το *argc*, που δηλώνει το πλήθος των ορισμάτων, και το *argv_capacity*, που δηλώνει τη διαθέσιμη χωρητικότητα του πίνακα. Επιπλέον, περιλαμβάνει τον πίνακα

redirects, στον οποίο αποθηκεύονται οι ανακατευθύνσεις που σχετίζονται με την εντολή, καθώς και τα αντίστοιχα πεδία για το πλήθος και τη χωρητικότητά του. Κάθε ανακατεύθυνση περιγράφεται από τη δομή *redirect*, η οποία περιλαμβάνει το πεδίο *type* και τη διαδρομή του αρχείου στο πεδίο *path*. Οι διαθέσιμοι τύποι ανακατεύθυνσης είναι *REDIRECT_INPUT* για την είσοδο, *REDIRECT_OUTPUT* για την έξοδο και *REDIRECT_APPEND* για την προσθήκη δεδομένων στο τέλος ενός αρχείου.

Για παράδειγμα, όταν ο χρήστης εισάγει τη γραμμή `ls -l`, ο parser δημιουργεί ένα *pipeline* που περιέχει μία εντολή. Στον πίνακα *argv* αποθηκεύονται τα στοιχεία "*ls*" (στο *argv[0]*) και "*-l*" (στο *argv[1]*), ενώ το *argc* έχει τιμή 2. Αντίστοιχα, για τη γραμμή `ls > output.txt`, ο parser αποθηκεύει την εντολή και τα ορίσματά της, ενώ παράλληλα καταγράφει μία ανακατεύθυνση τύπου *REDIRECT_OUTPUT* με διαδρομή "*output.txt*". Τα δεδομένα αυτά επιτρέπουν στο υπόλοιπο πρόγραμμα να γνωρίζει ότι η έξοδος της εντολής πρέπει να κατευθυνθεί στο συγκεκριμένο αρχείο. Ο parser, ωστόσο, δεν εκτελεί την εντολή ούτε πραγματοποιεί ο ίδιος την ανακατεύθυνση.

Η δομή *pipeline* επιτρέπει επίσης την αναπαράσταση γραμμών που περιέχουν περισσότερες από μία εντολές. Για παράδειγμα, στη γραμμή `ls | grep ".txt"`, ο parser μπορεί να αναπαραστήσει τις δύο εντολές με τη σειρά που εμφανίζονται, αποθηκεύοντας τα ορίσματά τους σε ξεχωριστές δομές *command*. Το πεδίο *command_count* προσδιορίζει το πλήθος των εντολών που περιλαμβάνονται στο *pipeline*. Το υπόλοιπο πρόγραμμα μπορεί στη συνέχεια να αξιοποιήσει αυτή τη δομή για να συνδέσει την έξοδο της πρώτης εντολής με την είσοδο της δεύτερης. Αντίστοιχα, στη γραμμή `sleep 10 &`, το πεδίο *background* μπορεί να χρησιμοποιηθεί από το τμήμα εκτέλεσης για να προσδιορίσει ότι η εντολή ζητήθηκε να εκτελεστεί στο παρασκήνιο.

Για παράδειγμα, ο παρακάτω κώδικας δείχνει πώς μπορεί να αναλυθεί μια γραμμή εντολών και να προσπελαστούν τα αποτελέσματα:

```
int main(void)
{
    const char *line = "ls -l > output.txt";
    struct pipeline pipeline = {0};

    int result = parse_line(line, &pipeline);

    if (result == PARSE_SUCCESS) {
        printf("command count: %u\n", pipeline.command_count);
        if (pipeline.command_count > 0) {
            struct command *cmd = &pipeline.commands[0]; // 0 holds the 1st command

            printf("Command name: %s\n", cmd->argv[0]);
            printf("Argument count: %u\n", cmd->argc);
            printf("Redirect count: %zu\n", cmd->redirect_count);
```

```
            if (cmd->redirect_count > 0) {
                printf("File for first redirection: %s\n", cmd->redirects[0].path);
            }
        }
    } else {
        printf("Parse error: %s\n", parse_error_string(result));
    }
    pipeline_free(&pipeline);
    return 0;
}
```

3.4. Βασική επανάληψη

Ο φλοιός περιλαμβάνει έναν ατέρμονα βρόγχο, στον οποίο σε κάθε επανάληψη διαβάζει μια γραμμή εντολής, την αναλύει, και την εκτελεί:

Επανέλαβε:

1. Εκτύπωση προτροπής εισόδου.
2. Διάβασμα και ανάλυση της γραμμής εισόδου.
3. Χειρισμός εντολής.
4. Επιστροφή στο βήμα 1.

Ο παραπάνω βρόχος μπορεί να διακοπεί τερματίζοντας τη διεργασία φλοιού με την ειδική εντολή *exit*.

Ο βασικός βρόγχος του φλοιού υλοποιείται στη συνάρτηση `main()`. Σε κάθε επανάληψη, ο φλοιός τυπώνει την προτροπή εισόδου, διαβάζει μια γραμμή εισόδου, την αναλύει χρησιμοποιώντας τη συνάρτηση `parse_line()`, την εκτελεί χρησιμοποιώντας τη συνάρτηση `execute_pipeline()`, καθαρίζει τη δομή *pipeline*, και τέλος επιστρέφει στην αρχή.

Ο βασικός μας στόχος, είναι να υλοποιήσουμε τη συνάρτηση `execute_pipeline()`, καθώς και όποια άλλη βοηθητική συνάρτηση απαιτείται.

3.5. Βήματα υλοποίησης

Θα υλοποιήσουμε την εργασία σταδιακά, έτσι ώστε να μειώσουμε την πολυπλοκότητα της υλοποίησης. Κάθε υπο-ενότητα που ακολουθεί, παρουσιάζει την υλοποίηση μιας λειτουργίας τη φορά.

3.5.1. Βήμα 1: Υλοποίηση ενσωματωμένων εντολών

Κάποιες εντολές θεωρούνται ενσωματωμένες, δηλαδή υποστηρίζονται απευθείας από το πρόγραμμα φλοιού και δεν απαιτείται η εκκίνηση ενός ξεχωριστού προγράμματος. Δύο παραδείγματα τέτοιων εντολών αποτελούν οι *exit* και *cd*. Η εντολή *exit* τερματίζει το πρόγραμμα φλοιού. Η εντολή *cd* αλλάζει τον τρέχων φάκελο εργασίας.

Σε πρώτη φάση θα υλοποιήσουμε αυτές τις δύο ενσωματωμένες εντολές. Αρχικά πρέπει να ελέγξουμε αν το όνομα της εντολής, το οποίο θα βρούμε στη μεταβλητή *argv[0]* της δομής *command*, ταιριάζει με κάποιο από τα *exit* ή *cd*. Μπορούμε στη συνέχεια να υλοποιήσουμε τις δύο εντολές ως εξής:

Εντολή exit. Η εντολή *exit* λέει στον φλοιό μας ότι πρέπει να τερματίσει. Για το σκοπό αυτό, πρώτα καθαρίζει το *pipeline* με χρήση της `pipeline_free()` και στη συνέχεια εκτελεί την κλήση συστήματος `exit(0)` για να ζητήσει τον τερματισμό του φλοιού.

Εντολή cd. Η εντολή *cd* αλλάζει των τρέχων φάκελο της διεργασίας του φλοιού μας. Δέχεται μια προαιρετική παράμετρο. Εφόσον ο χρήστης δεν έχει δώσει παράμετρο στη *cd*, τότε η *cd* αλλάζει τον τρέχων φάκελο στον ιδιωτικό φάκελο εργασίας του χρήστη (home directory). Μπορούμε να βρούμε αυτόν τον φάκελο χρησιμοποιώντας τη συνάρτηση `getenv()` ως εξής:

```
char *home = getenv(“HOME”);
```

Προσοχή: **<u>δε πρέπει</u>** να αποδεσμεύσουμε τον δείκτη *home* όταν δε τον χρειαζόμαστε άλλο (δηλαδή να καλέσουμε `free(home)`), καθώς δείχνει σε στατική μνήμη.

3.5.2. Βήμα 2: Υποστήριξη μη ενσωματωμένης εντολής στο προσκήνιο

Όταν θέλουμε να εκτελέσουμε ένα ξεχωριστό πρόγραμμα, πρέπει να δημιουργήσουμε μια ξεχωριστή διεργασία στα πλαίσια της οποίας θα εκτελεστεί το πρόγραμμα. Για το σκοπό αυτό, μπορούμε να χρησιμοποιήσουμε τις κλήσεις συστήματος `fork()`, `execvp()`, και `wait()` (ή `waitpid()`).

Κλήση συστήματος fork(): δημιουργεί μια νέα διεργασία που είναι αντίγραφο της γονικής διεργασίας με εξαίρεση ότι έχει το δικό της αντίγραφο μνήμης, κωδικό διεργασίας και δεικτών σε διαμοιραζομένους πόρους (π.χ. περιγραφείς αρχείων). Εάν η κλήση είναι επιτυχής, η εντολή που ακολουθεί τη `fork()` εκτελείται τόσο από τη γονική όσο και από τη νέα διεργασία. Στη γονική διεργασία, η συνάρτηση `fork()` επιστρέφει τον κωδικό της νέας διεργασίας, ενώ στη νέα διεργασία η `fork()` επιστρέφει την τιμή μηδέν.

Κλήση συστήματος execvp(char *file, char *argv[]): Αλλάζει το πρόγραμμα που εκτελεί η διεργασία. Το όρισμα *file* είναι το όνομα του αρχείου που περιέχει το νέο πρόγραμμα που θα εκτελεστεί. Ο πίνακας *argv* είναι μία λίστα από συμβολοσειρές παραμέτρων (μπορούμε να βρούμε το όνομα του εκτελέσιμου της πρώτης εντολής στη θέση *pipeline->commands[0].argv[0]*, και αντίστοιχα τον πίνακα με τις παραμέτρους της πρώτης εντολής στη δομή *pipeline->commands[0].argv.*

Κλήση συστήματος wait()/waitpid(): Χρησιμοποιείται από μία διεργασία για να αποκλειστεί (να μπλοκάρει) μέχρι ο πυρήνας να σηματοδοτήσει τη συνέχεια της εκτέλεσης. Για παράδειγμα, όταν ένα από τα παιδιά έχει τερματιστεί, η `wait()` επιστρέφει στη γονική την παράμετρο που περιέχει την κατάσταση τερματισμού του παιδιού. Αν θέλουμε να περιμένουμε τον τερματισμό *μιας συγκεκριμένης* διεργασίας με αναγνωριστικό *pid*, τότε μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `waitpid(pid, NULL, 0)`. Η κλήση `pid = waitpid(pid, NULL, WNOHANG)` επιστρέφει αμέσως (χωρίς να μπλοκάρει) το PID της διεργασίας που

ολοκληρώθηκε (η τιμή επιστροφής θα ισούται με την τιμή της μεταβλητής *pid*), ή έναν αριθμό <= 0 σε περίπτωση που δεν άλλαξε η κατάσταση της διεργασίας. Στην υλοποίησή μας μπορούμε να χρησιμοποιήσουμε όποια από τις παραπάνω συναρτήσεις μας βολεύει περισσότερο.

3.5.3. Βήμα 3: Υποστήριξη μη ενσωματωμένης εντολής στο παρασκήνιο

Ο φλοιός μπορεί να εκτελέσει μια εντολή στο παρασκήνιο, δηλαδή να ξεκινήσει την εντολή και στη συνέχεια να δεχτεί μια νέα εντολή χωρίς να περιμένει την προηγούμενη να ολοκληρωθεί. Για να ζητήσουμε από το φλοιό να εκτελέσει μια εντολή στο παρασκήνιο, τοποθετούμε το χαρακτήρα '*&*' στο τέλος της γραμμής εντολής:

```
$ command argument1 argument2 … &
```

Σε αυτή την περίπτωση ο φλοιός εκτελεί τα εξής βήματα:

1. Δημιουργία νέας διεργασίας.
2. Εκτέλεση της εντολής command στη διεργασία παιδί (νέα).
3. Εισαγωγή του PID της νέας διεργασίας στον πίνακα jobs.
4. Επιστροφή στην αρχή του βρόγχου.

Κάθε φορά που ο φλοιός δημιουργεί μια διεργασία παρασκηνίου, είναι σημαντικό να την εισάγουμε στον πίνακα *jobs*. Για το σκοπό αυτό μπορούμε να χρησιμοποιήσουμε τη βοηθητική συνάρτηση `add_bg_job()`, την οποία πρέπει να επεκτείνουμε ώστε να εισάγει στην πρώτη διαθέσιμη θέση του πίνακα *jobs* το PID της νέας διεργασίας παρασκηνίου. Πώς όμως ο φλοιός μπορεί να μάθει ότι μια διεργασία παρασκηνίου ολοκληρώθηκε ώστε να καθαρίσει τους πόρους της; Η λειτουργία αυτή μπορεί πολύ εύκολα να υλοποιηθεί με τη χρήση σημάτων, όπως θα δούμε παρακάτω.

Όταν τερματίζεται η εκτέλεση μιας διεργασίας παιδιού, ο πυρήνας αποστέλλει στη γονική διεργασία το σήμα *SIGCHLD*. Μπορούμε να χρησιμοποιήσουμε άρα αυτό το σήμα για να μάθουμε ότι μια εντολή παρασκηνίου ολοκληρώθηκε και στη συνέχεια να αποδεσμεύσουμε τους πόρους της. Για τον σκοπό αυτό, εγκαθιστούμε έναν χειριστή σήματος (*signal handler)*, δηλαδή μια δική μας συνάρτηση που εκτελείται όταν παραληφθεί το συγκεκριμένο σήμα. Μέσα στον χειριστή μπορούμε να χρησιμοποιήσουμε τη συνάρτηση `waitpid()` με κατάλληλες παραμέτρους, ώστε να συλλέξουμε την κατάσταση των διεργασιών παιδιών που έχουν τερματιστεί και να αποτρέψουμε τη δημιουργία διεργασιών-ζόμπι (zombie processes).

Για την εγκατάσταση του χειριστή χρησιμοποιούμε τη συνάρτηση `sigaction()`. Για παράδειγμα, για να εγκαταστήσουμε τον χειριστή `handle_sigchld()` για το σήμα `SIGCHLD`, μπορούμε να γράψουμε τον ακόλουθο κώδικα στην αρχή της συνάρτησης `main()`:

```
struct sigaction chld_action;
...
memset(&chld_action, 0, sizeof(chld_action));
chld_action.sa_handler = handle_sigchld;
sigemptyset(&chld_action.sa_mask);
chld_action.sa_flags = SA_RESTART;
```

```
if (sigaction(SIGCHLD, &chld_action, NULL) == -1) {
        perror("sigaction");
}
```

Το *SA_RESTART* είναι μια σημαία που μπορούμε να ορίσουμε στο πεδίο *sa_flags* της δομής *struct sigaction*. Ζητά από τον πυρήνα να επανεκκινεί αυτόματα ορισμένες κλήσεις συστήματος που διακόπτονται από την άφιξη ενός σήματος, αφού ολοκληρωθεί η εκτέλεση του χειριστή του.

Η συνάρτηση `handle_sigchld()` πρέπει να οριστεί ξεχωριστά και να καλεί την `reap_bg_jobs()`. Επειδή ο χειριστής εκτελείται ασύγχρονα και μπορεί να διακόψει το πρόγραμμα σε οποιοδήποτε σημείο, μέσα σε αυτόν επιτρέπονται μόνο λειτουργίες ασφαλείς για σήματα. Συγκεκριμένα, **δεν επιτρέπεται** η χρήση των `printf()`, `malloc()` και `free()`.

Η `reap_bg_jobs()` πρέπει να διατρέχει τον πίνακα *jobs* και, για κάθε ενεργή διεργασία, να καλεί τη `waitpid(pid, &status, WNOHANG)`. Η σημαία *WNOHANG* είναι απαραίτητη ώστε ο φλοιός να μην μπλοκάρει όσο υπάρχουν διεργασίες που εκτελούνται ακόμη. Επειδή το σήμα *SIGCHLD* δεν μπαίνει σε ουρά, ένα μόνο σήμα μπορεί να αντιστοιχεί σε περισσότερες από μία διεργασίες που τερματίστηκαν, γι' αυτό πρέπει να ελέγχονται **όλες** οι θέσεις του πίνακα. Αν η `waitpid()` επιστρέψει θετικό `PID`, η διεργασία αφαιρείται από τον πίνακα με την `remove_bg_job()`.

3.5.4. Βήμα 4: Χειρισμός του σήματος SIGINT

Όταν ο χρήστης πατήσει **Ctrl+C**, το τερματικό στέλνει το σήμα *SIGINT* σε **όλες** τις διεργασίες της ομάδας προσκηνίου. Επειδή ο φλοιός και οι διεργασίες παιδιά ανήκουν στην ίδια ομάδα, χωρίς ειδικό χειρισμό το σήμα θα τερμάτιζε τον ίδιο το φλοιό, καθώς και κάθε διεργασία παρασκηνίου που ξεκίνησε από αυτόν. Η επιθυμητή συμπεριφορά είναι διαφορετική:

- Ο φλοιός **δεν** τερματίζεται όταν πατηθεί Ctrl+C, αλλά συνεχίζει να εκτελείται και εμφανίζει ξανά το την προτροπή εισόδου
- Μια εντολή που εκτελείται στο **προσκήνιο** τερματίζεται κανονικά από το *SIGINT*.
- Οι εντολές που εκτελούνται στο **παρασκήνιο** δεν επηρεάζονται από το Ctrl+C.

Για να το πετύχουμε, αρκεί να αξιοποιήσουμε δύο ιδιότητες των σημάτων:

1. Αν μια διεργασία ορίσει το *SIGINT* ως *SIG_IGN* (αγνόηση), η ρύθμιση αυτή **κληρονομείται** από τις διεργασίες παιδιά που δημιουργεί με τη `fork()` και **διατηρείται** μετά την `execvp()`.
2. Αν ορίσουμε το *SIGINT* ως *SIG_DFL* (προεπιλεγμένη ενέργεια), η διεργασία τερματίζεται όταν λάβει το σήμα.

Άρα ο φλοιός αγνοεί το *SIGINT* στην αρχή της `main()`, και μόνο οι διεργασίες προσκηνίου επαναφέρουν την προεπιλεγμένη συμπεριφορά πριν εκτελέσουν την εντολή. Οι διεργασίες παρασκηνίου κληρονομούν την αγνόηση και δεν τερματίζονται. Για την εγκατάσταση χρησιμοποιούμε ξανά τη `sigaction()`:

```
struct sigaction int_action;
```

```
...
memset(&int_action, 0, sizeof(int_action));
int_action.sa_handler = SIG_IGN;
sigemptyset(&int_action.sa_mask);

if (sigaction(SIGINT, &int_action, NULL) == -1) {
    perror("sigaction");
    return 1;
}
```

Στη διεργασία παιδί, **πριν** την κλήση της `execvp()`, και μόνο όταν η εντολή εκτελείται στο προσκήνιο, επαναφέρουμε την προεπιλεγμένη συμπεριφορά:

```
// just before execvp:
struct sigaction dfl;
memset(&dfl, 0, sizeof(dfl));
dfl.sa_handler = SIG_DFL;
sigemptyset(&dfl.sa_mask);
sigaction(SIGINT, &dfl, NULL);
// call execvp here.
```

3.5.5. Βήμα 5: Υποστήριξη ανακατεύθυνσης εισόδου/εξόδου

Κάθε πρόγραμμα που ξεκινάει ο φλοιός έχει ανοικτά τρία αρχεία, το αρχείο προκαθορισμένης εισόδου (*stdin*, περιγραφέας 0), το αρχείο προκαθορισμένης εξόδου (*stdout*, περιγραφέας 1), και το αρχείο προκαθορισμένης εξόδου σφαλμάτων (*stderr*, περιγραφέας 2). Το αρχείο *stdin* αντιστοιχεί στο πληκτρολόγιο, ενώ τα άλλα δύο αντιστοιχούν στην οθόνη. Γι' αυτό το λόγο, όταν μια εντολή τυπώνει ένα μήνυμα, αυτό εμφανίζεται στην οθόνη.

Μπορούμε να ωστόσο να αλλάξουμε την προκαθορισμένη είσοδο, την προκαθορισμένη έξοδο, και την προκαθορισμένη έξοδο σφαλμάτων, ώστε να χρησιμοποιούν κανονικά αρχεία, αντί για το πληκτρολόγιο ή την οθόνη. Αυτή η αλλαγή ονομάζεται ανακατεύθυνση εισόδου/εξόδου. Στην παρούσα εργασία δε θα ασχοληθούμε περαιτέρω με την ανακατεύθυνση της προκαθορισμένης εξόδου σφαλμάτων.

Οι βασικοί τελεστές που θα θέλαμε να υποστηρίζει ο φλοιός για ανακατεύθυνση εισόδου/εξόδου είναι οι παρακάτω:

Τελεστής	**Λειτουργία**
`< αρχείο`	Η τυπική είσοδος της εντολής προέρχεται από το αρχείο.
`> αρχείο`	Η τυπική έξοδος γράφεται στο αρχείο, αντικαθιστώντας τα προηγούμενα περιεχόμενά του.

Για παράδειγμα, η εντολή:

```
ls -l > files.txt
```

τυπώνει τα αποτελέσματα στο αρχείο files.txt και όχι στην οθόνη.

Ο φλοιός μπορεί να ανακατευθύνει την Ε/Ε τροποποιώντας του περιγραφείς αρχείων της διεργασίας παιδιού. Ειδικότερα, ο φλοιός μπορεί να αλλάξει τους περιγραφείς αρχείων του παιδιού, έτσι ώστε να διαβάζει και να γράφει κανονικά αρχεία αντί για το πληκτρολόγιο και την οθόνη. Η κλήση συστήματός `close()` μπορεί να κλείσει ένα ανοιχτό αρχείο, συμπεριλαμβανομένων των *stdin*, *stdout*, και *stderr*. Επιπροσθέτως, η κλήση `open()` πάντα χρησιμοποιεί τη μικρότερη δυνατή θέση στον πίνακα περιγραφέων αρχείων, ενώ η κλήση συστήματος `dup2()` αντιγράφει έναν περιγραφέα σε έναν άλλον:

```
fid = open("foo", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(fid,STDOUT_FILENO); // copy fid on position 1. Use STDIN_FILENO for input.
close(fid); // we do not need fid any more (we can access the file through 1).
```

Ο κώδικας στο παραπάνω παράδειγμα, ανοίγει το αρχείο με όνομα *foo* για γράψιμο (αν δεν υπάρχει το δημιουργεί – παράμετρος *O_CREAT*, ενώ αν υπάρχει το μηδενίζει – παράμετρος *O_TRUNC*), και αντιγράφει τον περιγραφέα αρχείου, *fid*, στη θέση 1 που αντιστοιχεί στο αρχείο *stdout*. Έτσι, οι χαρακτήρες που γράφονται από τη διεργασία στο *stdout* αποθηκεύονται στο αρχείο *foo* αντί να εμφανίζονται στην οθόνη. Με αντίστοιχο τρόπο μπορούμε να ανακατευθύνουμε την προκαθορισμένη είσοδο (*stdin*, περιγραφέας 0). Προσοχή, η ανακατεύθυνση θα πρέπει να υλοποιηθεί μετά την κλήση της `fork()`, και πριν την κλήση της `execvp()`.

Ο αναλυτής parser αναλαμβάνει να εντοπίζει τις λειτουργίες ανακατεύθυνσης που ζητάει ο χρήστης. Η μεταβλητή *pipeline->commands[0].redirect_count* περιλαμβάνει τον αριθμό των ανακατευθύνσεων για την πρώτη εντολή. Για παράδειγμα, η τιμή της είναι 0 για την εντολή `minsh$ ls`, 1 για την εντολή `minsh$ ls > file`, και 2 για την εντολή `cat < file.in > file.out`. Οι πληροφορίες της 1η ανακατεύθυνσης αποθηκεύονται στη δομή `pipeline->commands[0].redirects[0]`, ενώ της 2ης ανακατεύθυνσης αποθηκεύονται στη δομή `pipeline->commands[0].redirects[1]`.

Για διευκόλυνση, ο φλοιός μας δεν απαιτείται να υποστηρίζει παραπάνω από μία ανακατευθύνσεις.

3.5.6. Βήμα 6: Υποστήριξη σωλήνωσης

Μια σωλήνωση (pipe) συνδέει την έξοδο μιας διεργασίας με την είσοδο μιας άλλης. Μπορούμε να έχουμε δύο ή περισσότερες διεργασίες που συνδέονται με σωλήνωση, και όλες εκτελούνται ταυτόχρονα από το σύστημα.

Για παράδειγμα:

```
$ ls -l | grep ".txt"
```

Η εντολή `ls -l` παράγει μια λίστα αρχείων. Η έξοδός της διοχετεύεται στην είσοδο της *grep*, η οποία εμφανίζει μόνο τις γραμμές που περιέχουν την κατάληξη *.txt*.

Μια διεργασία που επιθυμεί να δημιουργήσει σωλήνωση καλεί την κλήση συστήματος `pipe()`. Εφόσον η `fork()` δημιουργεί παιδί με αντίγραφο του πίνακα ανοιχτών αρχείων της γονικής διεργασίας, το παιδί

κληρονομεί τη σωλήνωση από τη γονική. Τότε, ο γονέας και το παιδί μπορούν να επικοινωνήσουν μέσω των κατάλληλων περιγραφέων. Για παράδειγμα:

```
int pipeID[2];
…
pipe(pideID);
if (fork() == 0) { /* διεργασία παιδί */
close(pipeID[1]);
read(pipeID[0], childBuf, len);
…
} else { /* γονική διεργασία */
close(pipeID[0]);
write(pipeID[1], msgToChild, len);
…
}
```

Για διευκόλυνση, δεν απαιτείται ο φλοιός μας να υποστηρίζει ταυτόχρονα ανακατεύθυνση εισόδου/εξόδου και σωληνώσεις.

3.5.7. Βήμα 7: Ενημέρωση του αρχείου SUBMIT.md

Ενημερώνουμε κατάλληλα το αρχείο `SUBMIT.md`. Αρχικά γράφουμε τα ονόματα και τους αριθμούς μητρώου των μελών της ομάδας. Στη συνέχεια περιγράφουμε σύντομα την υλοποίησή μας. Η περιγραφή μπορεί να περιλαμβάνει τον τρόπο με τον οποίο υαλοποιήσαμε τις λειτουργίες που υποστηρίζει ο φλοιός μας, καθώς και τις βασικές δομές δεδομένων και συναρτήσεις που χρησιμοποιήσαμε. Επιπλέον, περιγράφει τις περιπτώσεις που χρησιμοποιήσαμε για να δοκιμάσουμε τον κώδικά μας.

3.5.8. Βήμα 8: Εκτύπωση του τρέχων φακέλου (μπόνους 5%)

Συνήθως, η προτροπή εισόδου ενός φλοιού περιλαμβάνει πληροφορία για τον τρέχων φάκελο, δηλαδή τον φάκελο στον οποίο βρίσκεται ο φλοιός. Ο τρέχων φάκελος αλλάζει κάθε φορά όταν ο χρήστης εκτελεί την εντολή *cd*. Για να συμπεριλάβουμε αυτή την πληροφορία, πρέπει να διατηρήσουμε σε κάποια μεταβλητή (έστω *char *cwd*) το αλφαριθμητικό του τρέχων φακέλου και να το τυπώσουμε στη συνάρτηση `print_prompt()` με την παρακάτω μορφή:

```
printf("minish:%s$ ", cwd);
```

Το βήμα αυτό είναι προαιρετικό και θα μας δώσει 5% μπόνους στη βαθμολογία της άσκησης.

3.5.9. Βήμα 9: Ταυτόχρονη υποστήριξη πολλαπλών εντολών, σωλήνωσης, και ανακατεύθυνσης (μπόνους 15%)

Ένα πρόγραμμα φλοιού, όπως το bash, υποστηρίζει πολλαπλές σωληνώσεις και ανακατευθύνσεις, όπως:

```
$ ls | grep txt | sort
```

```
$ cat file.txt | sort > sorted.txt
```

```
$ cat < file.in > file.out
```

Με τη βοήθεια του parser μπορούμε να προσθέσουμε στο minsh υποστήριξη για πολλαπλές σωληνώσεις, πολλαπλές ανακατευθύνσεις, καθώς και για ταυτόχρονη σωλήνωση και ανακατεύθυνση.

Το βήμα αυτό είναι προαιρετικό και θα μας δώσει 15% μπόνους στη βαθμολογία της άσκησης.

4. Παράδοση

Μπορείτε να υποβάλετε την εργασία σας είτε ατομικά, είτε σε ομάδες των δύο. Δεν απαιτείται να δηλώσετε κάπου τις ομάδες, αλλά θα πρέπει να περιλαμβάνονται τα ονόματα των μελών στο αρχείο SUBMIT.md και οι αριθμοί μητρώου στο όνομα του αρχείου προς υποβολή. Είστε ελεύθεροι να υποβάλλετε κάθε εργασία με διαφορετική ομάδα.

Είναι σημαντικό να ακολουθήσετε τα επόμενα βήματα κατά γράμμα ώστε το αρχείο που θα παραδώσετε να έχει την αναμενόμενη μορφή και να μπορέσετε να βαθμολογηθείτε.

Ανοίξτε ένα τερματικό και μεταβείτε στο φάκελο που περιλαμβάνει τον φάκελο minsh-base, δηλαδή στο φάκελο που εκτελέσατε την εντολή `git clone` για να κατεβάσετε τον κώδικα της άσκησης. Εκτελώντας την εντολή `ls`, θα πρέπει να βλέπετε τον φάκελο minsh-base:

```
$ ls
```

```
… minsh-base …
```

Καθαρίστε το φάκελο build:

```
$ make clean
```

Εκτελέστε την ακόλουθη εντολή για να πακετάρετε τον φάκελο (αντικαταστήστε τα iis00000 με τους σωστούς αριθμούς μητρώου των μελών της ομάδας. Οι αριθμοί μητρώου χωρίζονται μεταξύ τους με το χαρακτήρα της παύλας):

```
$ tar cvfz iis00000-iis00000.tar.gz minsh-base
```

Έπειτα, συνδεθείτε στο openeclass και μεταβείτε στη σελίδα του μαθήματος. Από εκεί, επιλέξτε το μενού Εργασίες και στη συνέχεια την Εργασία 1: Υλοποίηση ενός απλού φλοιού (shell). Στη σελίδα που θα σας ανοίξει μπορείτε να υποβάλετε το αρχείο `iis00000-iis00000.tar.gz` που δημιουργήσατε στο προηγούμενο βήμα.

Προσοχή, αν και οι εργασίες είναι ομαδικές, θα πρέπει κάθε μέλος της ομάδας να υποβάλει την εργασία ξεχωριστά.